ETSI TS 104 146 V1.1.1 (2026-08)



TECHNICAL SPECIFICATION

Cyber Security (CYBER);
Quantum-Safe Cryptography (QSC);
Authenticated Quantum Safe Hybrid Key Establishment

Reference
DTS/CYBER-QSC-0030

Keywords
Quantum Safe Cryptography

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
ETSI Search & Browse Standards application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on ETSI deliver repository.

Users should be aware that the present document may be revised or have its status changed, this information is available in the Milestones listing.

If you find errors in the present document, please send your comments to the relevant service listed under Committee Support Staff.

If you find a security vulnerability in the present document, please report it through our Coordinated Vulnerability Disclosure (CVD) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.
No recommendation as to products and services or vendors is made or should be implied.
No representation or warranty is made that this deliverable is technically accurate or sufficient or conforms to any law and/or governmental rule and/or regulation and further, no representation or warranty is made of merchantability or fitness for any particular purpose or against infringement of intellectual property rights.
In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part of this document may be reproduced in any form, by any means and in any media, without the prior written authorization of ETSI and except as expressly permitted below.
By way of exception and when the document is a normative deliverable (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)), ETSI authorizes to reproduce and incorporate into products, services and technical documentation only those extracts (e.g. templates) that are strictly necessary for the technical implementation of the normative deliverable, to ensure compliance with the latter.
Nothing in this notice shall be construed as limiting any mandatory exceptions to copyright provided by applicable law.

© ETSI 2026.
All rights reserved.

Contents

Intellectual Property Rights 4
Foreword 4
Modal verbs terminology 4
Executive summary 4
Introduction 4
1 Scope 5
2 References 5
2.1 Normative references 5
2.2 Informative references 6
3 Definition of terms, symbols and abbreviations 6
3.1 Terms 6
3.2 Symbols 6
3.3 Abbreviations 7
4 Overview of Authenticated Quantum-Safe Hybrid Key Establishment 7
4.1 Motivation and Objectives 7
4.2 Use Cases and Applications Scenarios 7
4.3 Comparison with Existing Standards 8
5 Protocol Architecture 8
5.1 Functional Entities and Roles 8
5.2 Information Relationships (Reference Points) 8
6 Cryptographic Primitives 8
6.1 Notation 8
6.2 Hash Function 8
6.3 Elliptic Curve Diffie-Hellman (ECDH) 8
6.4 Post-Quantum Key Encapsulation Mechanisms (PQ KEMs) 8
6.5 Quantum-Based Cryptographic Component Integration 9
6.6 Message Authentication Code 9
6.7 Digital Signature Schemes (DSSs) 9
6.8 Key Derivation Functions (KDFs) 9
6.9 Authenticated Encryption with Associated Data (AEAD) 10
7 Protocol Specifications 10
7.1 General 10
7.1.1 Key Establishment Abstraction Framework 10
7.1.2 Authenticated Quantum-Safe Hybrid Key Establishment 12
7.1.3 Security Aspects 15
History 17

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)) may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI IPR online database.

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM**® and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

This Technical Specification (TS) has been produced by ETSI Technical Committee Cyber Security (CYBER).

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

Executive summary

The present document specifies a method for authenticated quantum-safe hybrid key establishment by securely combining quantum-safe cryptographic key-material with a quantum-safe authentication mechanism for any two communicating entities.

Introduction

Authenticated Quantum-Safe Hybrid Key Establishment (AQSHKE) is a secure cryptographic protocol that ensures an authenticated, confidential, and integrity-protected channel between any two communicating entities, even in the event of a Cryptographically Relevant Quantum Computer (CRQC). Such security guarantee is established using (optional) traditional asymmetric cryptography, post-quantum and symmetric cryptography, and quantum-based cryptographic components.

1 Scope

The present document specifies a method for establishing authenticated quantum-safe shared cryptographic key material from multiple sources of quantum-safe key material. Specifically, such key material can be derived from (optional) traditional cryptography, post-quantum and symmetric cryptography, and quantum-based cryptographic components with a post-quantum authentication mechanism.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

Referenced documents which are not found to be publicly available in the expected location might be found in the ETSI docbox.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents are necessary for the application of the present document.

[1] ETSI TS 103 744 (V1.2.2): "CYBER; Quantum-Safe Cryptography (QSC); Quantum-safe Hybrid Key Establishment ".

[2] ISO/IEC 18033-2:2006/Amd 2:2026: "Information technology — Security techniques — Encryption algorithms — Part 2: Asymmetric ciphers — Amendment 2".

[3] ETSI GS QKD 014 (V1.1.1): "Quantum Key Distribution (QKD); Protocol and data format of REST-based key delivery API".

[4] ETSI GS QKD 004 (V2.1.1): "Quantum Key Distribution (QKD); Application Interface".

[5] NIST SP 800-38B: "Recommendation for Block Cipher Modes of Operation: the CMAC Mode for Authentication".

[6] ISO/IEC 9797-1:2011: "Information technology – Security techniques – Message Authentication Codes (MACs) – Part 1: Mechanisms using a block cipher".

[7] FIPS 204: "Module-Lattice-Based Digital Signature Standard".

[8] FIPS 205: "Stateless Hash-Based Digital Signature Standard".

[9] FIPS 186-5: "Digital Signature Standard (DSS)".

[10] IETF RFC 5869: "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)".

[11] IETF RFC 5116: "An Interface and Algorithms for Authenticated Encryption".

[12] IETF RFC 9846: "The Transport Layer Security (TLS) Protocol Version 1.3".

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents may be useful in implementing an ETSI deliverable or add to the reader's understanding, but are not required for conformance to the present document.

[i.1] Buruaga, J.S., Bugler, A., Brito, J.P., Martin, V., Striecks, C. (2025): "Versatile quantum-safe hybrid key exchange and its application to MACsec". EPJ Quantum Technol. 12, 84 (2025). DOI: 10.1140/epjqt/s40507-025-00382-x.

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the terms given in ETSI TS 103 744 [1] and the following apply:

NOTE: Definitions were taken from the ETSI TEDDI database (https://webapp.etsi.org/Teddi/) if available.

(digital) certificate: public key of an entity, together with some other information, rendered unforgeable by digital signature with the private key of the certification authority which issued it

digital signature scheme: keyed asymmetric cryptographic scheme that is used to protect the authenticity of data

message authentication code: cryptographic check value that is used to provide data origin authentication and data integrity

nonce: arbitrary number that is generated for security purposes (such as an initialization vector) that is used only one time in any security session

Public Key Infrastructure (PKI): architecture, organization, techniques, practices, and procedures that collectively support the implementation and operation of a certificate-based public key cryptographic system, including the services established to issue, maintain, and revoke public key certificates

quantum-safe: not vulnerable to quantum computing attack

3.2 Symbols

For the purposes of the present document, the symbols given in ETSI TS 103 744 [1] and the following apply:

$\oplus$	A bitwise XOR operator
c	A ciphertext value
cert	A digital certificate
Δ	A list of authentication materials
K	A list of keys
H	The digest returned by a hash function
l	A label
M	A message
n	A cryptographic nonce
psk	A pre-shared key
SecState	A secret state (e.g. an optional symmetric pre-shared key only known to involved entities)
dsk	A private signing key for a digital signature scheme
σ	A digital signature
τ	An authentication tag created by MAC

vk	A public verification key for a digital signature scheme
⊥	An error symbol

3.3 Abbreviations

For the purposes of the present document, the abbreviations given in ETSI TS 103 744 [1] and the following apply:

AEAD	Authenticated Encryption with Associated Data
AQSHKE	Authenticated Quantum-Safe Hybrid Key Establishment
CRQC	Cryptographically Relevant Quantum Computers
DSS	Digital Signature Scheme
EUF-CMA	Existential Unforgeability under Chosen Message Attack
HAKE	Hybrid Authenticated Key Establishment
ITS	Information Theoretic Security
MAC	Message Authentication Code
PKI	Public Key Infrastructure
PQC	Post-Quantum Cryptography

4 Overview of Authenticated Quantum-Safe Hybrid Key Establishment

4.1 Motivation and Objectives

Public-key primitives are one of the main components of authenticated key establishment protocols. The development of Cryptographically Relevant Quantum Computers (CRQCs) threatens the security guarantees of many key establishment protocols based on public-key primitives. Authenticated Quantum-Safe Hybrid Key Establishment (AQSHKE) aims at establishing authenticated quantum-safe shared key material from key material coming based on multiple quantum-safe cryptographic primitives while mitigating the threat of CRQCs.

The AQSHKE protocol objectives are:

- The protocol may be integrated into classical and quantum communication infrastructures.
- The protocol shall output authenticated quantum-safe shared key material for two entities.
- At least one quantum-safe Key Encapsulation Mechanism (KEM) in combination with traditional cryptography or one quantum-based cryptographic mechanism (i.e. based on Quantum-Key Distribution (QKD)) with a quantum-safe authentication mechanism may be integrated into the protocol.
- Any standardized quantum-safe KEM and any standardized digital signature scheme may be integrated into the protocol.
- Cryptographic key material via key-delivery interfaces in [3], [4] based on quantum cryptography may be integrated into the protocol for confidentiality.
- An optional symmetric key may be integrated into the protocol for confidentiality.
- Authentication shall be integrated via quantum-safe symmetric or asymmetric authentication mechanisms.

4.2 Use Cases and Applications Scenarios

AQSHKE ensures quantum-safe communication between any two entities in large-scale communication networks (particularly, with quantum-based cryptographic components available). The use of at least post-quantum (in combination with traditional) cryptography or quantum-based cryptography for confidentiality (such as QKD) and post-quantum cryptography for authentication in the AQSHKE protocol at least for the initial run is recommended. An external quantum-safe Public Key Infrastructure (PKI) is required in such case which is outside of the scope of the present document.

For small-scale communication networks (particularly, with quantum-based cryptographic components available), pre-shared key material may be additionally used for confidentiality. Moreover, pre-shared key material may be alternatively used for authentication instead of post-quantum digital signatures and a PKI.

Through incorporating symmetric keys (for confidentiality and authentication), AQSHKE may be capable of enhancing security for symmetric-key-only systems by mixing in post-quantum and/or quantum-cryptography-based key material.

4.3 Comparison with Existing Standards

The present document builds on ETSI TS 103 744 [1] and uses (parts of) its terms, symbols, abbreviations, functional entities and roles, and cryptographic primitives. The key establishment protocol in ETSI TS 103 744 [1] does neither include the authentication of any two entities nor the integrity of the key.

5 Protocol Architecture

5.1 Functional Entities and Roles

As in ETSI TS 103 744 [1], clause 5.1.

5.2 Information Relationships (Reference Points)

As in ETSI TS 103 744 [1], clause 5.2.

6 Cryptographic Primitives

6.1 Notation

As in ETSI TS 103 744 [1], clause 6.2.

6.2 Hash Function

As in ETSI TS 103 744 [1], clause 7.1.

In the present document, in clause 7, the hash function is used to provide a digest of context inputs.

6.3 Elliptic Curve Diffie-Hellman (ECDH)

As in ETSI TS 103 744 [1], clause 8.1.

In the present document, in clause 7, ECDH provides a (traditional) key-material source for confidentiality.

6.4 Post-Quantum Key Encapsulation Mechanisms (PQ KEMs)

As in ETSI TS 103 744 [1], clauses 7.6.1 and 7.6.2. Moreover, PQ KEMs from [2] may be used.

In the present document, in clause 7, the PQ KEM provides a key-material source for confidentiality.

6.5 Quantum-Based Cryptographic Component Integration

The interfaces from [3], clauses 5.3 and 5.4 shall be used: GetKey to retrieve one or more keys with key IDs each of length 128 or 256 bits and GetKeyWithKeyIDs(keyIDs) to retrieve keys on specific key IDs each of length 128 or 256 bits. The received keys via GetKey and GetKeyWithKeyIDs(keyIDs) shall be statistically indistinguishable from the uniform distribution given access to all public information. Each key ID in the list keyIDs as output of GetKey shall be only used once and shall only be queried once via GetKeyWithKeyIDs.

Alternatively, the interface from [4], clause 6.1, may be used: GET_KEY(Key_Stream_ID, NULL) or GET_KEY(Key_Stream_ID, index) to retrieve keys on specific key stream IDs each of length 128 or 256 bits. In this case, implementations shall ensure that the key-delivery interface is protected with authentication, confidentiality, and integrity equivalent to those mandated by [3] (e.g. mutual authentication of both endpoints, physical isolation, or OS-level access restriction). Each index shall be only used once and shall only be queried once. Implementations using interfaces from [4] shall document how these properties are achieved.

In the present document, in clause 7, the quantum-based cryptographic component provides a key-material source for confidentiality.

6.6 Message Authentication Code

The present clause specifies the basic properties of a Message Authentication Code (MAC) as a tuple of probabilistic algorithms $(MAC.Auth, MAC.Verify)$ associated with a key space (determined via the outcome of an associated key-generation algorithm) and a message space:

- $MAC.Auth(mk, m)$ takes as input a key mk from the key space and a message m from the message space, and outputs a tag τ, or $\perp$ an error indicator.
- $MAC.Verify(mk, m, \tau)$, a deterministic algorithm, takes as input a key mk, a message m, and a tag τ, and outputs 1 (meaning valid) or 0 (meaning invalid), or $\perp$ an error indicator.

The MAC CMAC [5], [6] is recommended.

In the present document, in clause 7, the MAC ensures key confirmation and (optional) authentication with a pre-shared key.

6.7 Digital Signature Schemes (DSSs)

The present clause specifies the basic properties of a Digital Signature Scheme (DSS) as a tuple of probabilistic algorithms $(DSS.Sign, DSS.Verify)$ associated with a message space and a private-public key-pair space (determined via the outcome of an associated key-generation algorithm):

- $DSS.Sign(dsk, m)$ takes a digital signature key dsk and a message m from the message space as input, and outputs a signature σ, or $\perp$ an error indicator.
- $DSS.Verify(vk, m, \sigma)$, a deterministic algorithm, takes as input a public verification key vk, a message m, and a signature σ, and outputs 1 (meaning valid) or 0 (meaning invalid), or $\perp$ an error indicator.

The current list of approved post-quantum DSSs consists of the FIPS-approved ML-DSA [7] and SLH-DSA [8]. Alternatively, the FIPS-approved traditional signatures schemes from [9] may be used.

In the present document, in clause 7, the PQ DSS ensures authentication.

6.8 Key Derivation Functions (KDFs)

As in ETSI TS 103 744 [1], clauses 7.4.1, 7.4.2, and 7.4.4. where the length parameter length of the derived keying material shall be set to 16 or 32 octets and omitted in the following. The calling interfaces for the extract and expand functions are as defined in [10], sections 2.2. and 2.3, respectively, where the length parameter L of the derived keying material in the expand function shall be set to 16 or 32 octets and omitted in the following.

The length parameters shall be of the same bit length as the length of the keys in clause 6.5.

In the present document, in clause 7, the KDF outputs key material.

6.9 Authenticated Encryption with Associated Data (AEAD)

The present clause specifies the basic interfaces of an Authenticated Encryption with Associated Data (AEAD) as given in [11] as a tuple of deterministic algorithms $(AEAD.E, AEAD.D)$ with a key, nonce, message, and associated data spaces except that the nonce and the associated data are added to the ciphertext c as the first and the second part, respectively:

- $AEAD.E(k_{AE}, n, p, a)$ takes a private key k_{AE} from the key space, a nonce n from the nonce space, a plaintext p from the message space, and associated data a from the associated data space as input, and outputs a ciphertext c, or $\perp$ an error indicator.
- $AEAD.D(k_{AE}, c)$, a deterministic algorithm, takes as input a key k_{AE}, a ciphertext c, and outputs a message m, or $\perp$ an error indicator.

AEAD schemes with key size 128 bits from [11] shall be used.

The keys, the nonces, and the associated data shall be defined as in [12], section 5.2. The abstraction functions $DeriveTK(k_{IHTS})$ and $DeriveTK(k_{RHTS})$ used in the present document's clause 7 shall map the input keys k_{IHTS} and k_{RHTS} to client_write_key and server_write_key as defined in [12], section 5.2, respectively. The abstraction function $AEAD.CONT(l)$ shall, on input a length parameter l, return distinct nonces $(n_1, \ldots, n_l)$ and associated data $(a_1, \ldots, a_l)$ as defined in [12], section 5.2.

In the present document, in clause 7, AEAD ensures confidentiality of the digital certificates.

7 Protocol Specifications

7.1 General

7.1.1 Key Establishment Abstraction Framework

The present clause specifies a functional framework for combining quantum-safe cryptographic key material and a post-quantum digital-signature scheme into a secure protocol. The framework is based on the VMuckle quantum-safe Hybrid Authenticated Key Exchange (HAKE) protocol presented in [i.1].

The design rational is as follows. An ephemeral key exchange phase is run to establish an intermediate key and authentication key material. This key material is fed into the authentication and key confirmation phase with the result of an authenticated shared key and a secure (updated) state. The shared key may be used for further data encryption while the secure state may be used for a future protocol run.

The parameters and function calls shall be as follows:

- (dsk_i, vk_i) and (dsk_r, vk_r) are initiator A and responder B signing and verification keys, respectively. The corresponding initiator and responder certificates are $cert_i$ and $cert_r$, respectively. Verification keys are included in the associated certificates. The keys and certificates come from an external PKI and may be valid.
- psk_a is an (optional) pre-shared key for authentication only available to A and B. If unused, psk_a shall be set to $\perp$.
- $SecState$ is an (optional) secure state shared by A and B with length 16 or 32 octets (depending on the KDF length parameter). If unused, $SecState$ shall be set to $\perp$.
- $InitKeyGen()$ is a key generation function which returns a list of secret keys SK and a list of public keys P.
- MA is an octet string sent from A to B containing an encoding of one or more public keys. MA may include session negotiation information.

- MB is an octet string sent from B to A containing an encoding of one or more response values. MB may include session negotiation information.
- MC is an octet string sent from A to B containing an encoding of one or more AEAD ciphertexts.
- MD is an octet string sent from B to A containing an encoding of one or more AEAD ciphertexts.
- $\mathrm{RespEphFunc}(SecState, MA)$ is a responder ephemeral-key function which takes a secure state $SecState$ and an initial message MA as input, and returns either a list of shared ephemeral keys $ephK$ and a response value R, or an error indicator $(\perp, \perp)$.
- $\mathrm{InitEphFunc}(SK, SecState, MB)$ is an initiator ephemeral-key function which takes a list of secret keys SK, a secure state $SecState$ and a response message MB as input and returns either a list of shared ephemeral keys $ephK$, or an error indicator $\perp$.
- $\mathrm{InterDer}(ephK, psk_a, SecState)$ is an intermediate key derivation function which takes a list of ephemeral keys ephK, a pre-shared authentication key psk_a and a secure state $SecState$ as input, and returns either a derivation key list DK, an initiator authentication key list AK_i and a responder authentication key list AK_r.
- $\mathrm{RespAuthFunc}(AK_r, psk_a, dsk_r, cert_r)$ is a responder authentication function which takes a list of responder authentication keys AK_r, a pre-shared authentication key psk_a, a digital signature key dsk_r and a certificate $cert_r$ as input, and returns either a list of authentication related elements Δ_r, or an error indicator $\perp$. If $\mathrm{RespAuthFunc}$ returns an error indicator, B shall respond with an error message and terminate the process with an error message. If A receives an error message, A shall terminate the process with an error message.
- $\mathrm{InitVerFunc}(AK_r, psk_a, MC)$ is an initiator verification function which takes a responder authentication key list AK_r, a pre-shared key psk_a and a message MC as input, and either indicates a valid responder authentication process (i.e. by returning a 1) or returns an error indicator $\perp$. If $\mathrm{InitVerFunc}$ returns an error indicator, A shall respond with an error message and terminate the process.
- $\mathrm{FinalDer}(DK)$ is a final key derivation function which takes a list of derivation keys DK as input and returns a list of output keys K and an updated secure state $SecState'$ where K shall consists of two keys with 128 or 256 bits each in bit length. The first key may be used for the initiator-related payload data while the second key may be used for the responder-related payload data; however, payload data encryption after the key exchange execution ended is out of scope of the present document.
- $\mathrm{InitAuthFunc}(AK_i, psk_a, dsk_i, cert_i)$ is an initiator authentication function which takes a list of initiator authentication keys AK_i, a pre-shared key psk_a, a digital signature key dsk_i and a certificate $cert_i$ as input, and returns either a list of authentication related elements Δ_i, or an error indicator $\perp$. If $\mathrm{InitAuthFunc}$ returns an error indicator, A shall respond with an error message and terminate the process. If B receives an error message, B shall terminate the process with an error message.
- $\mathrm{RespVerFunc}(AK_i, psk_a, MD)$ is a responder verification function which takes an initiator authentication key list AK_i, a pre-shared key psk_a and a message MD as input, and either indicates a valid responder authentication process (i.e. by returning a 1) or returns an error indicator $\perp$. If $\mathrm{RespVerFunc}$ returns an error indicator, B shall terminate the process.

The following mapping of label-context inputs shall be used:

Label	Context Input	Label	Context Input
l_0	"derive key 0"	l_1	"derive key 1"
l_2	"derive key 2"	l_3	"derive key 3"
l_4	"derived"	l_5	"i hs traffic"
l_6	"r hs traffic"	l_7	"i finished"
l_8	"r finished"	l_9	"i ap traffic"
l_{10}	"r ap traffic"	l_{11}	"secstate"
l_{12}	"Responder CertificateVerify"	l_{13}	"Initiator CertificateVerify"
H_ε	""	H_0	$H("")$
H_1	$H(MA \parallel MB)$	H_2	$H(MA \parallel MB \parallel c_{m3})$
H_3	$H(MA \parallel MB \parallel c_{m3} \parallel c_{m4})$	H_4	$H(MA \parallel MB \parallel c_{m3} \parallel c_{m4} \parallel c_{m5})$
H_5	$H(MA \parallel MB \parallel c_{m3} \parallel c_{m4} \parallel c_{m5} \parallel c_{m6})$	H_6	$H(MA \parallel MB \parallel c_{m3} \parallel c_{m4} \parallel c_{m5} \parallel c_{m6} \parallel c_{m7})$

Functional mappings between the naming convention and the calling abstractions are provided in clauses 7.1.2.

An AQSHKE protocol comprises two phases: an ephemeral key exchange phase and an authentication with key confirmation phase.

Figure 1 illustrates the protocol phases. The protocol succeeds if RespAuthFunc,InitVerFunc, InitAuthFunc, and RespVerFunc do not return the ⊥ symbol.

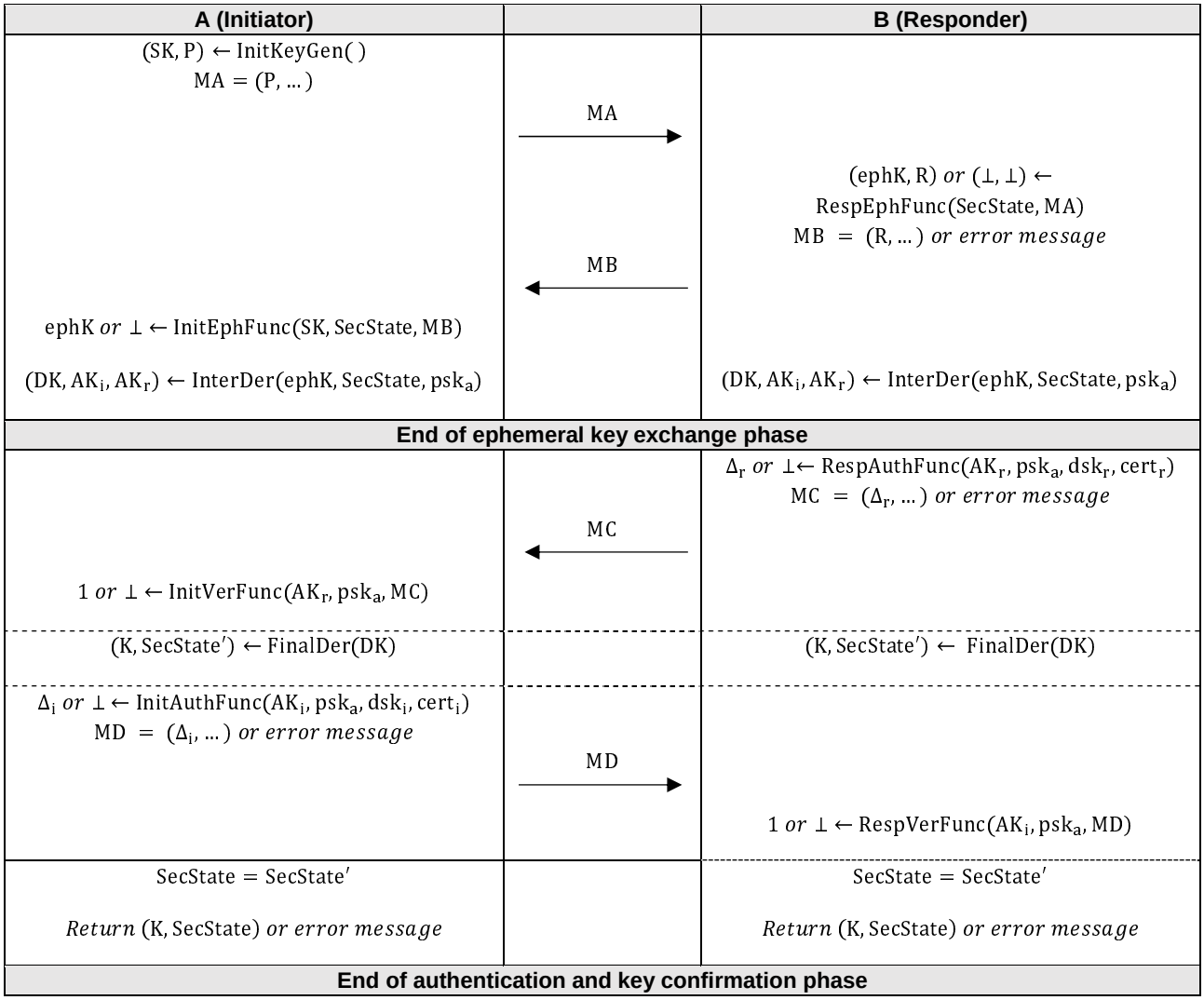


Figure 1: Protocol phases abstraction with authenticated key K

7.1.2 Authenticated Quantum-Safe Hybrid Key Establishment

This clause specifies AQSHKE by a mapping of the key establishment abstraction interfaces to ECDHE, PQ KEM, ETSI GS QKD 014 [3] interfaces, KDFs, MAC, and PQ DSS methods. The values qk_{IATS}, qk_{RATS}, and qk_{STATE} may be optional and are set to 0 in such case. Each entity shall keep a log of all context inputs, which shall be updated whenever the relevant inputs become available, and which may be accessed during the execution of any functions by that entity. At least the key generation of the PQ KEM (line 2) or the key generation of GetKey() (line 12) may yield valid values (sk_{pq}, pk_{pq}) or $(keyIDs, ss_q)$, respectively.

InitKeyGen( ) shall be as follows:

1) $(sk_c, pk_c) = ECDHEKeyGen()$
2) $(sk_{pq}, pk_{pq}) = PQKeyGen()$
3) *The value* n_i *shall be a random 32-octet string*

4) $SK = (sk_c, sk_{pq})$

5) $P = (pk_c, pk_{pq}, n_i)$

6) *Return* (SK, P)

RespEphFunc(SecState, MA) shall be as follows:

7) $(P, \dots) = MA$

8) $(pk_c, pk_{pq}, n_i) = P$

9) $(c_c, ss_c) = \text{ECDHEEncaps}(pk_c)$

10) $(c_{pq}, ss_{pq}) = \text{PQEncaps}(pk_{pq})$

11) *If* $ss_{pq} = \perp$ *and* SecState $= \perp$ *return* $(\perp, \perp)$

12) $(\text{keyIDs}, ss_q, qk_{IATS}, qk_{RATS}, qk_{STATE}) = \text{GetKey}()$

13) *The value* n_r *shall be a random 32-octet string*

14) $\text{ephK} = (ss_c, ss_{pq}, ss_q, qk_{IATS}, qk_{RATS}, qk_{STATE})$

15) $R = (c_c, c_{pq}, n_r, \text{keyIDs})$

16) *Return* (ephK, R)

InitEphFunc(SK, SecState, MB) shall be as follows:

17) $(R, \dots) = MB$

18) $(c_c, c_{pq}, n_r, \text{keyIDs}) = R$

19) $(sk_c, sk_{pq}) = SK$

20) $ss_c = \text{ECDHEDecaps}(c_c, sk_c)$

21) $ss_{pq} = \text{PQDecaps}(c_{pq}, sk_{pq})$

22) *If* $ss_{pq} = \perp$ *and* SecState $= \perp$ *return* $\perp$

23) $(ss_q, qk_{IATS}, qk_{RATS}, qk_{STATE}) = \text{GetKeyWithKeyIDs}(\text{keyIDs})$

24) $\text{ephK} = (ss_c, ss_{pq}, ss_q, qk_{IATS}, qk_{RATS}, qk_{STATE})$

25) *Return* ephK

InterDer(ephK, psk_a, SecState) shall be as follows:

26) $(ss_c, ss_{pq}, ss_q, qk_{IATS}, qk_{RATS}, qk_{STATE}) = \text{ephK}$

27) $k_0 = \text{HKDF}(\text{SecState}, 0, l_0 \parallel H_0)$

28) $k_1 = \text{HKDF}(k_0, ss_c, l_1 \parallel H_0)$

29) $k_2 = \text{HKDF}(k_1, ss_{pq}, l_2 \parallel H_0)$

30) $k_3 = \text{HKDF}(k_2, ss_q, l_3 \parallel H_0)$

31) $k_{dHS} = \text{HKDF.Expand}(k_3, l_4 \parallel H_0)$

32) $k_{IHTS} = \text{HKDF.Expand}(k_3, l_5 \parallel H_1)$

33) $k_{RHTS} = \text{HKDF.Expand}(k_3, l_6 \parallel H_1)$

34) $\mathrm{fk_i} = \mathrm{HKDF}(\mathrm{k_{IHTS}}, \mathrm{psk_a}, \mathrm{l_7} \parallel \mathrm{H_\varepsilon})$

35) $\mathrm{fk_r} = \mathrm{HKDF}(\mathrm{k_{RHTS}}, \mathrm{psk_a}, \mathrm{l_8} \parallel \mathrm{H_\varepsilon})$

36) $\mathrm{tk_{ihs}} = \mathrm{DeriveTK}(\mathrm{k_{IHTS}})$

37) $\mathrm{tk_{rhs}} = \mathrm{DeriveTK}(\mathrm{k_{RHTS}})$

38) $\mathrm{DK} = (\mathrm{k_{dHS}}, \mathrm{qk_{IATS}}, \mathrm{qk_{RATS}}, \mathrm{qk_{STATE}})$

39) $\mathrm{AK_i} = (\mathrm{fk_i}, \mathrm{tk_{ihs}})$

40) $\mathrm{AK_r} = (\mathrm{fk_r}, \mathrm{tk_{rhs}})$

41) *Return* $(\mathrm{DK}, \mathrm{AK_i}, \mathrm{AK_r})$

$\mathrm{RespAuthFunc}(\mathrm{AK_r}, \mathrm{psk_a}, \mathrm{dsk_r}, \mathrm{cert_r})$ shall be as follows:

42) $(\mathrm{fk_r}, \mathrm{tk_{rhs}}) = \mathrm{AK_r}$

43) $(\mathrm{n_1}, \mathrm{n_2}, \mathrm{n_3}, \mathrm{a_1}, \mathrm{a_2}, \mathrm{a_3}) = \mathrm{AEAD.\,CONT}(3)$

44) $\mathrm{c_{m3}} = \mathrm{AEAD.\,E}(\mathrm{tk_{rhs}}, \mathrm{n_1}, \mathrm{cert_r}, \mathrm{a_1})$

45) *If* $\mathrm{dsk_r} \neq \perp$ *then* $\sigma_\mathrm{r} = \mathrm{DSS.\,Sign}(\mathrm{dsk_r}, \mathrm{l_{12}} \parallel \mathrm{H_2})$

46) *If* $\sigma_\mathrm{r} = \perp$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

47) $\mathrm{c_{m4}} = \mathrm{AEAD.\,E}(\mathrm{tk_{rhs}}, \mathrm{n_2}, \sigma_\mathrm{r}, \mathrm{a_2})$

48) $\tau_\mathrm{r} = \mathrm{MAC.\,Auth}(\mathrm{fk_r}, \mathrm{H_3})$

49) $\mathrm{c_{m5}} = \mathrm{AEAD.\,E}(\mathrm{tk_{rhs}}, \mathrm{n_3}, \tau_\mathrm{r}, \mathrm{a_3})$

50) $\Delta_\mathrm{r} = (\mathrm{c_{m3}}, \mathrm{c_{m4}}, \mathrm{c_{m5}})$

51) *Return* Δ_r

$\mathrm{InitVerFunc}(\mathrm{AK_r}, \mathrm{psk_a}, \mathrm{MC})$ shall be as follows:

52) $(\mathrm{fk_r}, \mathrm{tk_{rhs}}) = \mathrm{AK_r}$

53) $(\Delta_\mathrm{r}, \ldots) = \mathrm{MC}$

54) $(\mathrm{c_{m3}}, \mathrm{c_{m4}}, \mathrm{c_{m5}}) = \Delta_r$

55) $\mathrm{cert_r} = \mathrm{AEAD.\,D}(\mathrm{tk_{rhs}}, \mathrm{c_{m3}})$

56) *If* $\mathrm{cert_r}$ *is not a valid certificate for* $\mathrm{vk_r}$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

57) $\sigma_\mathrm{r} = \mathrm{AEAD.\,D}(\mathrm{tk_{rhs}}, \mathrm{c_{m4}})$

58) *If* $\sigma_\mathrm{r} = \perp$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

59) *If* $\mathrm{DSS.\,Verify}(\mathrm{vk_r}, \mathrm{l_{12}} \parallel \mathrm{H_2}, \sigma_\mathrm{r}) \neq 1$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

60) $\tau_\mathrm{r} = \mathrm{AEAD.\,D}(\mathrm{tk_{rhs}}, \mathrm{c_{m5}})$

61) *If* $\mathrm{MAC.\,Verify}(\mathrm{fk_r}, \mathrm{H_3}, \tau_\mathrm{r}) \neq 1$ *return* $\perp$

62) *Return* 1

$\mathrm{FinalDer}(\mathrm{DK})$ shall be as follows:

63) $(\mathrm{k_{dHS}}, \mathrm{qk_{IATS}}, \mathrm{qk_{RATS}}, \mathrm{qk_{STATE}}) = \mathrm{DK}$

64) $\mathrm{k_{MS}} = \mathrm{HKDF.\,Extract}(\mathrm{k_{dHS}}, 0)$

65) $\mathrm{IATS} = \mathrm{HKDF.\,Expand}(\mathrm{k_{MS}}, \mathrm{l_9} \parallel \mathrm{H_4}) \oplus \mathrm{qk_{IATS}}$

66) $\mathrm{RATS} = \mathrm{HKDF.\,Expand}(\mathrm{k_{MS}}, \mathrm{l_{10}} \parallel \mathrm{H_4}) \oplus \mathrm{qk_{RATS}}$

67) $\mathrm{SecState}' = \mathrm{HKDF.\,Expand}(\mathrm{k_{MS}}, \mathrm{l_{11}} \parallel \mathrm{H_4}) \oplus \mathrm{qk_{STATE}}$

68) $\mathrm{K} = (\mathrm{IATS}, \mathrm{RATS})$

69) *Return* $(\mathrm{K}, \mathrm{SecState}')$

$\mathrm{InitAuthFunc}(\mathrm{AK_i}, \mathrm{psk_a}, \mathrm{dsk_i}, cert_i)$ shall be as follows:

70) $(\mathrm{fk_i}, \mathrm{tk_{ihs}}) = \mathrm{AK_i}$

71) $(\mathrm{n_4}, \mathrm{n_5}, \mathrm{n_6}, \mathrm{a_4}, \mathrm{a_5}, \mathrm{a_6}) = \mathrm{AEAD.\,CONT}(3)$

72) $\mathrm{c_{m6}} = \mathrm{AEAD.\,E}(\mathrm{tk_{ihs}}, \mathrm{n_4}, \mathrm{cert_i}, \mathrm{a_4})$

73) *If* $\mathrm{dsk_i} \neq \perp$ *then* $\sigma_\mathrm{i} = \mathrm{DSS.\,Sign}(\mathrm{dsk_i}, \mathrm{l_{13}} \parallel \mathrm{H_5})$

74) *If* $\sigma_\mathrm{i} = \perp$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

75) $\mathrm{c_{m7}} = \mathrm{AEAD.\,E}(\mathrm{tk_{ihs}}, \mathrm{n_5}, \sigma_\mathrm{i}, \mathrm{a_5})$

76) $\tau_\mathrm{i} = \mathrm{MAC.\,Auth}(\mathrm{fk_i}, \mathrm{H_6})$

77) $\mathrm{c_{m8}} = \mathrm{AEAD.\,E}(\mathrm{tk_{ihs}}, \mathrm{n_6}, \tau_\mathrm{i}, \mathrm{a_6})$

78) $\Delta_\mathrm{i} = (\mathrm{c_{m6}}, \mathrm{c_{m7}}, \mathrm{c_{m8}})$

79) *Return* Δ_i

$\mathrm{RespVerFunc}(\mathrm{AK_i}, \mathrm{psk_a}, \mathrm{MD})$ shall be as follows:

80) $(\mathrm{fk_i}, \mathrm{tk_{ihs}}) = \mathrm{AK_i}$

81) $(\Delta_\mathrm{i}, \ldots) = \mathrm{MD}$

82) $(\mathrm{c_{m6}}, \mathrm{c_{m7}}, \mathrm{c_{m8}}) = \Delta_\mathrm{i}$

83) $\mathrm{cert_i} = \mathrm{AEAD.\,D}(\mathrm{tk_{ihs}}, \mathrm{c_{m6}})$

84) *If* $\mathrm{cert_i}$ *is not a valid certificate for* $\mathrm{vk_i}$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

85) $\sigma_\mathrm{i} = \mathrm{AEAD.\,D}(\mathrm{tk_{ihs}}, \mathrm{c_{m7}})$

86) *If* $\sigma_\mathrm{i} = \perp$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

87) *If* $\mathrm{DSS.\,Verify}(\mathrm{vk_i}, \mathrm{l_{13}} \parallel \mathrm{H_5}, \sigma_\mathrm{i}) \neq 1$ *and* $\mathrm{psk_a} = \perp$ *return* $\perp$

88) $\tau_i = \mathrm{AEAD.\,D}(\mathrm{tk_{ihs}}, \mathrm{c_{m8}})$

89) *If* $\mathrm{MAC.\,Verify}(\mathrm{fk_i}, \mathrm{H_6}, \tau_i) \neq 1$ *return* $\perp$

90) *Return* 1

7.1.3 Security Aspects

Under the assumptions that the PQ KEM is at least IND-CPA secure, the PQ DSS is EUF-CMA secure, the quantum-based cryptographic component yields a secure key $\mathrm{ss_q}$, the MAC is a secure MAC, and the KDF is a secure dual pseudo-random function, the protocol in clauses 7.1.1 and 7.1.2 is a secure HAKE protocol in the sense of [i.1] with the optional quantum-cryptography-based keys $\mathrm{qk_{IATS}}$, $\mathrm{qk_{RATS}}$, and $\mathrm{qk_{STATE}}$ in lines 65-67 set to all-zero.

The XOR operations in lines 65-67 do not negatively impact the security claims from [i.1] for the final shared key material K and the secure state SecState. If used, the security guarantees of the quantum-cryptography-based keys qk_{IATS}, qk_{RATS}, and qk_{STATE} are preserved for the final shared key material K and the secure state SecState. If the quantum-based cryptographic component yields information-theoretically secure keys, then the final shared key material K and the secure state SecState retain information-theoretic security guarantees after the protocol succeeded.

Secure implementation measures are out of scope of the present document.

History

Version	Date	Status
V1.1.1	August 2026	Publication